Trabalho de Criptoanálise

Grupo 14

Recuperação e decifração de mensagem confidencial
interceptada pelo Biuro Szyfrów

Integrantes:
Julia Brenda Souza da Rosa
Eduarda dos Santos da Silva

Disciplina: Algoritmos e Estrutura de Dados III
Professor: Ulisses Brisolara Corrêa
Universidade Federal de Pelotas (UFPEL)

28/10/2026

1. Introdução

Este relatório documenta o processo completo de criptoanálise realizado para recuperar e decifrar uma mensagem confidencial, conforme o cenário proposto: um criptoanalista do Biuro Szyfrów que intercepta uma mensagem protegida por múltiplas camadas de criptografia. Foram fornecidos quatro arquivos:

- **message.en**: a mensagem secreta cifrada
- **key_for_message.en**: a chave de decifração da mensagem, protegida por criptografia de chave pública (RSA)
- **key_public.en**: a chave pública RSA correspondente, protegida por criptografia simétrica (senha)
- **key_for_rsa_public.hash**: hash criptográfico da senha de acesso à chave pública

Nenhum algoritmo, modo de operação, tamanho de chave ou codificação foi informado previamente, tudo precisou ser deduzido ou descoberto por tentativa sistemática, exceto o início da senha (“gu”), fornecido como pista inicial. O processo foi dividido em quatro etapas sequenciais, cada uma dependente do resultado da anterior:

1. **Etapa 1:** descobrir a senha a partir do hash fornecido
2. **Etapa 2:** usar a senha para decifrar key_public.en e obter a chave pública RSA
3. **Etapa 3:** fatorar o módulo RSA, recuperar a chave privada e decifrar key_for_message.en
4. **Etapa 4:** usar o resultado da Etapa 3 para decifrar message.en e obter a mensagem final

2. Procedimentos executados

2.1. Etapa 1: Descoberta da senha

O arquivo *key_for_rsa_public.hash* contém uma string de 64 caracteres hexadecimais, equivalente a 32 bytes, tamanho de digest compatível com vários algoritmos de hash de 256 bits (SHA-256, SHA3-256, Keccak-256, BLAKE2s, BLAKE2b truncado, SHA-512/256, SM3, GOST Streebog-256, entre outros). A pista fornecida indicava que a senha começa com “gu”.

Passo a passo executado:

1. Confirmar o tamanho do hash: `wc -c` no arquivo e contagem de caracteres hex (64 → 32 bytes).
2. Montar dicionários combinando wordlists do sistema (português, inglês, cracklib) e o wordlist público *rockyou.txt* (14,3 milhões de senhas reais vazadas), filtrando por prefixo “gu” (script `crack_hash.py`).
3. Aplicar mutações de senha (maiúsculas/minúsculas, sufixos numéricos e símbolos comuns) e regras do hashcat (best64, rockyou-30000, dive, até 99 mil regras por palavra).
4. Rodar força bruta com máscara (hashcat -a 3) fixando o prefixo “gu”/“Gu”/“GU” e variando o sufixo em três conjuntos de caracteres (minúsculas+dígitos, +maiúsculas, ASCII completo com símbolos), para vários tamanhos totais de senha.
5. Testar a mesma cobertura contra 7 algoritmos de hash de 256 bits diferentes, não apenas SHA-256.
6. Usar pistas de contexto (nome do professor, disciplina, universidade, regionalismos do Rio Grande do Sul, temática do Biuro Szyfrów em polonês) para gerar candidatos temáticos adicionais.
7. Estender a força bruta usando GPU (Google Colab, notebook `colab_quebra_senha.py`) para cobrir tamanhos maiores de forma viável em tempo.

Registro das execuções (força bruta e dicionário). A tabela a seguir resume, em ordem, as tentativas rodadas na máquina local (hashcat sobre CPU/GPU) e, por último, na GPU do Google Colab. A coluna “combinações” é o tamanho do espaço de busca coberto em cada teste; os tempos são de execução real.

#	Teste	Combinações	Tempo
1	Força bruta “gu” + 1 a 6 (minúsculas+maiúsculas+dígitos)	56,8 bilhões	~1min44s
2	Hipótese de “\n” no final da senha (mesmo espaço)	56,8 bilhões	~1min29s
3	Força bruta “gu” + 1 a 6 com símbolos (ASCII completo)	742 bilhões	~19 min
4	Tamanhos 7 e 8, só minúsculas	8 bi + 208 bi	segundos a minutos
5	Tamanho 7, minúsculas+maiúsculas+dígitos	3,5 trilhões	~1h32min
6	Dicionário pt-BR (palavras começando com “gu”)	1.936	segundos
6b	Dicionário pt-BR + regra best66	127.776	segundos
6c	Dicionário pt-BR + regra dive	191.025.120	segundos

7	Dicionário polonês (palavras começando com “gu”)	414	segundos
7b	Dicionário polonês + regra dive	40.849.380	segundos
8	**“gu” + 10 minúsculas, SENHA ENCONTRADA (GPU Tesla T4, Colab)**	**141 trilhões**	**52 s**

Antes de partir para o hashcat, a mesma varredura por força bruta foi prototipada em Python puro (script `crack_hash.py`): o espaço “gu” + 6 minúsculas+dígitos (2,24 bilhões de combinações) levou ~51 min só de CPU, e “gu” + 5 minúsculas+maiúsculas+dígitos (931 milhões) levou ~19 min. Esses tempos deixaram claro o porquê de migrar para o hashcat com aceleração por GPU.

Execução que encontrou a senha. O comando abaixo, rodado na GPU Tesla T4 do Google Colab (notebook `colab_quebra_senha.py`, apêndice 5.3), testa a máscara “gu” seguida de 10 letras minúsculas:

```
hashcat -m 1400 -a 3 hash.txt "gu?l?l?l?l?l?l?l?l?l" -O -w 3 -d 1 \
    --session=gpu_colab \
    --restore-file-path=.../hashcat_progresso/gpu_colab.restore
```

Notebook completo no Google Colab:
`https://colab.research.google.com/drive/1Ew62xDPIvDcw4M2rQCV2qTLXA4SIf62N`

Observação sobre o processo. O espaço dessa máscara tem 26^10 ≈ 141 trilhões de combinações, mas o hashcat encontrou a senha em apenas 52 segundos, o indicador de progresso marcava só 0,09% do espaço percorrido no momento do acerto. Ou seja, a senha estava relativamente “cedo” na ordem em que o hashcat varre o espaço; não é que percorrer os 141 trilhões leve 52 segundos. O tempo real depende de onde a senha cai nessa ordem, e o pior caso (esgotar o espaço inteiro) ficaria na faixa de dias, coerente com as estimativas de tempo da seção 4.1.

Resultado: senha = guysteelexxx (hash SHA-256; encontrada na execução #8).

2.2. Etapa 2: Decifrar key_public.en

Antes de ter a senha, foi feita uma análise estrutural dos arquivos cifrados (tamanhos e cabeçalhos), que não depende de nenhuma senha:

Arquivo	Ciphertext	Dedução
message.en	1520 bytes (múltiplo de 16)	Cifra de bloco 128-bit com padding (CBC/ECB)
key_public.en	157 bytes (primo, não múltiplo de 8 nem 16)	Sem padding de bloco, modo stream (CTR/CFB/OFB)
key_for_message.en	48 bytes = 384 bits	Tamanho típico de ciphertext RSA cru, módulo provavelmente de 384 bits

A conclusão de que key_public.en usa um modo stream (sem padding) foi decisiva: ela elimina de imediato as combinações CBC/ECB (que exigiriam um ciphertext múltiplo do tamanho de bloco) e restringe a busca a cifras CTR/CFB/OFB/fluxo. Com a senha em mãos, o script `etapa2_decifrar_key_public.sh` testa sistematicamente ~37 cifras desse tipo (AES, ChaCha20, RC4, Blowfish, CAST5, DES, SM4, Camellia, ARIA, SEED) combinadas com KDF legado ou PBKDF2 e três opções de digest (sha256/sha1/md5), verificando se a saída contém um cabeçalho PEM de chave pública válido.

Algoritmo/modo identificado: AES-256-CTR, KDF legado EVP_BytesToKey (sem PBKDF2), digest SHA-256 (padrão)

2.3. Etapa 3: Fatorar o RSA e decifrar key_for_message.en

Com a chave pública em mãos (módulo N de 382 bits e expoente e extraídos do PEM), o script `etapa3_fatorar_rsa.py` tenta fatorar N em ordem crescente de custo computacional: consulta ao banco público factordb.com, divisão por tentativa, fatoração de Fermat (eficiente quando os primos p e q são próximos, um erro clássico de geração de chave), Pollard's rho e Pollard p-1 (eficiente quando p-1 ou q-1 é suave). **Nenhuma dessas técnicas rápidas funcionou**: o factordb não tinha a fatoração, não há fatores pequenos, p e q não são próximos e p-1/q-1 não são suaves. Isso mostra que a chave, apesar de curta, não possui as fraquezas estruturais clássicas, os dois primos são equilibrados (191 bits cada) e aparentemente aleatórios.

A fatoração exigiu, então, um algoritmo de crivo de propósito geral. Tentou-se ainda PARI/GP (`factor()`, que abandonou o MPQS por “número grande demais”) e GMP-ECM com B1 = 43.000.000 (ineficaz, pois o ECM é lento para semiprimos equilibrados, ~13 h sem resultado). A solução veio com o `yafu v3.1.9` forçando o **SIQS** (Self-Initializing Quadratic Sieve) com `siqs(N)` em 12 threads, que fatorou N em aproximadamente 11 horas (39.946 s), coletando ~299 mil relações. Esse é exatamente o tipo de ataque que a própria mensagem final descreve: um módulo RSA pequeno o bastante para cair diante de um crivo em um desktop comum.

Após obter p, q e calcular phi(N) = (p-1)(q-1), o expoente privado d é obtido pelo inverso modular de e em phi(N). Com (n, d) recuperados, o ciphertext hexadecimal de key_for_message.en é decifrado via RSA (m = c^d mod n). O bloco resultante começa com `00 02` seguido de bytes de preenchimento não-nulos, um separador `00` e, só então, a mensagem, isto é, **padding PKCS#1 v1.5** (tipo 2). Removendo o padding, obtém-se a senha em ASCII usada para cifrar message.en.

Módulo N	884448668806657999081001449267985268759583045069232509539285795978980877251751417211616710762958318527583397028207 (382 bits)
Expoente e	65537 (0x10001)
Fatores p, q	p = 291327370720422398713870856388599859111885259610476212132 7 (191 bits) q = 303592713111544615183046519921027009563438984396525954944 1 (191 bits) Fatorado com yafu v3.1.9 (SIQS, 12 threads) em ~11 h (39.946 s).
Senha/texto recuperado	jackdongarra (bloco RSA com padding PKCS#1 v1.5; senha = bytes após o separador 0x00)

2.4. Etapa 4: Decifrar message.en

message.en é um arquivo binário puro (sem base64), com cabeçalho “Salted__” + salt de 8 bytes idêntico ao formato usado em key_public.en, ou seja, também foi cifrado por senha via OpenSSL. Como a Etapa 3 produz uma senha/texto (não uma chave bruta), essa senha é usada diretamente. A análise estrutural indicou um ciphertext múltiplo exato de 16 bytes (ao contrário de key_public.en), apontando para uma cifra de bloco com padding (CBC/ECB) em vez de stream. O script `etapa4_decifrar_message.sh` testa ~34 cifras de bloco (AES, DES/3DES, Blowfish, CAST5, Camellia, ARIA, SM4, SEED, RC2) x KDF legado/PBKDF2 x três digests, verificando automaticamente se a saída contém os nomes dos integrantes do grupo (garantido pelo enunciado).

Algoritmo/modo identificado: AES-256-CBC, KDF legado EVP_BytesToKey (sem PBKDF2), digest SHA-256

3. Resultados finais

3.1. Senha descoberta (Etapa 1)

guysteelexxx

Algoritmo de hash: SHA-256

3.2. Algoritmos e codificações identificados

Arquivo	Algoritmo/modo	Codificação
key_for_rsa_public.hash	SHA-256	hexadecimal
key_public.en	AES-256-CTR, KDF legado EVP_BytesToKey (sem PBKDF2), digest SHA-256 (padrão)	base64 (openssl enc)
key_for_message.en	RSA 382 bits, padding PKCS#1 v1.5	hexadecimal
message.en	AES-256-CBC, KDF legado EVP_BytesToKey (sem PBKDF2), digest SHA-256	binário (openssl enc)

3.3. Mensagem decifrada (completa)

```
MENSAGEM CONFIDENCIAL — GRUPO 14
============================================================

Destinatários autorizados: Julia Brenda Souza da Rosa, Eduarda dos Santos da Silva

Classificação: ULTRA SECRETO
Protocolo: Biuro Szyfrów — Operação Enigma 2026-2

Prezados agentes,

A operação de interceptação foi bem-sucedida. Os dados coletados
confirmam que o sistema de criptografia inimigo utilizou chaves
RSA de baixa entropia combinadas com AES-256 para proteger as
comunicações. Esta combinação, embora aparentemente segura,
possui vulnerabilidades que podem ser exploradas sistematicamente.

Nossos analistas identificaram que a chave pública RSA pode ser
fatorada em tempo viável utilizando o algoritmo GNFS (General
Number Field Sieve). A senha utilizada para proteger esta chave
foi derivada de um espaço de busca limitado, permitindo ataques
de força bruta contra o hash armazenado.

Recomendações operacionais:
  1. Utilize ferramentas especializadas para recuperação da senha.
  2. Aplique algoritmos de fatoração para a chave RSA.
  3. Documente todos os procedimentos para replicabilidade.

Este documento foi gerado exclusivamente para o grupo 14
(Julia Brenda Souza da Rosa, Eduarda dos Santos da Silva). Qualquer compartilhamento não autorizado será
detectado através da análise forense dos metadados embutidos.

Assinado digitalmente,
Biuro Szyfrów — Divisão de Criptoanálise
Data: 11/10/2026
============================================================
```

4. Caminhos tentados que não funcionaram

Diversas abordagens foram tentadas e eliminadas sistematicamente antes de se chegar aos resultados finais, nas três etapas de criptoanálise (descoberta da senha, decifração de key_public.en e fatoração do RSA). As três primeiras são detalhadas individualmente abaixo; em seguida, resumem-se as demais tentativas da Etapa 1 (4.4) e da Etapa 3 (4.5), sempre com a explicação técnica do motivo da falha.

4.1. Força bruta combinando tamanhos 10 a 12 em um único comando hashcat

O que foi tentado: rodar `hashcat --increment --increment-min=10 --increment-max=12` em uma única execução, assumindo que estender a cobertura de tamanho da senha de 10 para 12 caracteres seria uma extensão incremental barata.

Por que falhou (ficou impraticável): o espaço de busca cresce exponencialmente com o tamanho. Para o alfabeto minúsculas+dígitos (36 símbolos), o tamanho 10 tem 36^8 ≈ 2,8 trilhões de combinações (levou ~19 minutos numa GPU a 2,47 GH/s), mas o tamanho 11 tem 36^9 ≈ 101 trilhões (~11-12 horas) e o tamanho 12 tem 36^10 ≈ 3,6 quatrilhões (~17 dias), na MESMA GPU. Como o hashcat processa os tamanhos em ordem crescente dentro de um único comando, o tamanho 12 só começaria a ser testado depois de completar o 10 e o 11, tornando a execução conjunta inviável. A lição técnica é que o tempo de uma força bruta por máscara cresce multiplicativamente a cada caractere adicional do alfabeto (fator 36x por posição, nesse caso), então somar vários tamanhos “grandes” no mesmo comando não é uma extensão linear e pode facilmente ultrapassar o tempo disponível. A solução foi isolar cada tamanho em comandos separados e avaliar o custo individualmente antes de rodar.

4.2. Colisão de instâncias do hashcat ao rodar ataques em paralelo

O que foi tentado: disparar duas execuções do hashcat simultaneamente em segundo plano (uma testando o alfabeto minúsculas+dígitos e outra minúsculas+MAIÚSCULAS+dígitos) para paralelizar o trabalho e economizar tempo total.

Por que falhou: o hashcat usa, por padrão, um arquivo de lock de instância única (verificado pela mensagem de erro “Already an instance '/usr/bin/hashcat' running”). A segunda execução encerrou imediatamente com código de saída 1, sem processar nenhuma combinação, enquanto a primeira ainda estava rodando, o que, sem verificação cuidadosa dos logs, poderia ter sido confundido com uma faixa de senha genuinamente “esgotada sem sucesso” quando na verdade ela nunca chegou a ser testada. A lição técnica é que ferramentas de GPU/OpenCL frequentemente assumem acesso exclusivo ao dispositivo e não paralelizam automaticamente entre processos separados; a solução foi rodar as tentativas estritamente em sequência, sempre conferindo o log completo (não só o código de saída) antes de considerar uma faixa como “testada”.

4.3. Falso negativo na Etapa 2: bug do OpenSSL ao combinar -a/-A com -provider

O que foi tentado: depois de obter a senha (Etapa 1), o script de decifração de key_public.en combinava as flags `-a -A` (decodificação base64 embutida no próprio comando `openssl enc`)

com `-provider legacy -provider default` (necessário para habilitar cifras mais antigas como RC4/DES/Blowfish, movidas para um provider separado desde o OpenSSL 3.0).

Por que falhou: essa combinação específica de flags produz o erro silencioso "error reading input file" em todas as 259 combinações de cifra/KDF testadas - inclusive para cifras padrão como AES, que definitivamente não deveriam falhar. Como o script apenas verificava se o arquivo de saída continha uma chave PEM válida (sem checar o código de retorno do openssl), todas as tentativas simplesmente produziam arquivos vazios e eram silenciosamente contadas como "não bateu", quando na verdade nenhuma delas chegou a executar de fato. Isso é particularmente traiçoeiro: o resultado (0 sucessos) é indistinguível de uma senha errada, levando a se questionar a senha descoberta na Etapa 1 quando na verdade o problema era puramente operacional. A causa raiz parece ser uma incompatibilidade entre o parser de base64 embutido do `openssl enc` e o carregamento explícito de providers nesta versão (OpenSSL 3.0.13). A solução foi decodificar o base64 manualmente com `base64 -d` antes de invocar o `openssl enc` diretamente sobre o binário já decodificado (sem as flags -a/-A), o que eliminou o erro. A lição técnica, reforçada aqui, é sempre verificar o código de saída e o stderr de cada tentativa automatizada, não apenas o resultado esperado - um "não encontrado" pode ser um falso negativo por falha de execução, não por hipótese incorreta.

4.4. Resumo de outras tentativas sem sucesso (Etapa 1)

Além dos dois casos detalhados acima, as seguintes abordagens também foram testadas e eliminadas, sem apresentar uma falha técnica "interessante" em si (apenas não bateram com o hash-alvo):

- Dicionário rockyou.txt completo (14,3 milhões de senhas reais vazadas), filtrado por prefixo "gu", combinado com até 99 mil regras de mutação do hashcat (~3 bilhões de variações testadas).
- 7 algoritmos de hash de 256 bits diferentes (SHA-256, SHA3-256, Keccak-256, BLAKE2s, BLAKE2b truncado, SHA-512/256, SM3, GOST Streebog-256).
- Candidatos temáticos manuais em português, inglês, alemão e polonês relacionados ao contexto do enunciado (Biuro Szyfrów, nomes do grupo, nome do professor/disciplina/universidade, regionalismos do Rio Grande do Sul).
- Variações de encoding (senha com/sem quebra de linha antes do hash, UTF-8) e de hash truncado/duplo/salgado com o nome do arquivo.

4.5. Tentativas de fatoração que falharam (Etapa 3)

A fatoração do módulo RSA de 382 bits foi a parte mais custosa do trabalho. Antes de chegar à técnica que funcionou (SIQS), várias abordagens mais baratas foram tentadas e descartadas, cada uma por um motivo técnico específico, documentado abaixo:

- **factordb.com:** o módulo não constava no banco público de fatorações (status "C", composto não fatorado). Não é um módulo reaproveitado/conhecido.
- **Divisão por tentativa (até 2 milhões):** não há fatores pequenos, os dois primos têm 191 bits.

- **Fatoração de Fermat (5 milhões de iterações):** falhou porque p e q não são próximos entre si (|p – q| é da ordem de 10^56). Fermat só é eficiente quando os primos estão muito próximos de √N.
- **Pollard's rho e Pollard p–1:** ineficazes para fatores de 191 bits; o p–1 em particular exigiria que p–1 ou q–1 fosse “suave” (só fatores pequenos), o que não ocorre.
- **PARI/GP factor():** abandonou o crivo MPQS com a mensagem “number too big to be factored with MPQS, giving up”, a implementação do MPQS no PARI tem um teto de tamanho abaixo do nosso N.
- **GMP-ECM (B1 = 43.000.000, curvas ilimitadas):** rodou ~13 h sem achar fator. O método das curvas elípticas é rápido quando existe um fator relativamente pequeno, mas é ineficiente para semiprimos equilibrados (dois fatores do mesmo tamanho), que é justamente o nosso caso.
- **yafu com GNFS (tentativa inicial):** o binário pré-compilado do yafu não inclui o *lattice siever* externo (`gnfs-lasieve4I12e`) do projeto GGNFS; sem ele, a etapa de sieving do GNFS aborta (`ggnfs returned code 32512`). A saída foi forçar o SIQS, que é autocontido.

Conclusão: o módulo não é “fraco” no sentido clássico (sem primos próximos, sem suavidade, sem fator pequeno). Ele é simplesmente **curto demais** (382 bits) para os padrões atuais, e por isso cedeu a um crivo quadrático de propósito geral rodando poucas horas num computador comum.

5. Apêndice: Código-fonte completo

5.1. crack_hash.py

Etapa 1: ataque de dicionário em Python

```
#!/usr/bin/env python3
"""
Etapa 1 - tentativa de quebrar a senha por dicionario.
O hash tem 64 hex (256 bits), entao chutamos SHA-256. Sabemos que a
senha comeca com "gu", entao juntamos umas wordlists do sistema, filtramos
pelo prefixo, aplicamos algumas mutacoes e comparamos o hash. Se o SHA-256
nao bater, testa sha3_256 e blake2s tambem.
"""
import hashlib

TARGET_HASH = "befb7036fb77a8517e95d9dbb797031850fce2ecdb988e96d7f040a2608e87e4"
KNOWN_PREFIX = "gu"

WORDLIST_FILES = [
    "/usr/share/dict/portuguese",
    "/usr/share/dict/brazilian",
    "/usr/share/dict/cracklib-small",
    "/usr/share/john/password.lst",
    "/usr/share/dict/american-english",
]

# chutes nossos ligados ao tema do trabalho
THEMATIC_WORDS = [
    "guerra", "guerreiro", "guarda", "guardiao", "guardia", "guardian",
    "guarani", "gustavo", "guaraci", "guaxinim", "gurgel", "gula",
    "guzik", "gu14", "grupo14", "gu2026",
]


def load_candidates():
    seen = set()
    candidates = []

    def add(w):
        w = w.strip()
        if w and w not in seen:
            seen.add(w)
            candidates.append(w)

    for path in WORDLIST_FILES:
        try:
            with open(path, "r", encoding="latin-1", errors="ignore") as f:
                for line in f:
                    add(line.rstrip("\n"))
        except FileNotFoundError:
            print(f"[aviso] wordlist nao encontrada: {path}")

    for w in THEMATIC_WORDS:
        add(w)

    return candidates


def mutate(word):
    # capitaliza/maiusculas + sufixos comuns (numeros, ano, simbolo)
    base_forms = {word, word.lower(), word.upper(), word.capitalize()}
    suffixes = ["", "1", "12", "123", "!", "@", "#", "2024", "2025", "2026",
                "14", "#14"]
    for b in base_forms:
        for s in suffixes:
            yield b + s


def hash_fns():
    return {
        "sha256": lambda s: hashlib.sha256(s.encode("utf-8")).hexdigest(),
        "sha3_256": lambda s: hashlib.sha3_256(s.encode("utf-8")).hexdigest(),
        "blake2s": lambda s: hashlib.blake2s(s.encode("utf-8")).hexdigest(),
    }


def main():
    print(f"[*] hash alvo: {TARGET_HASH}")
    print(f"[*] tamanho do hash: {len(TARGET_HASH)} hex chars -> {len(TARGET_HASH)//2} bytes")
    print(f"[*] prefixo conhecido da senha: '{KNOWN_PREFIX}'")
```

```
    raw_candidates = load_candidates()
    print(f"[*] total de palavras carregadas (antes do filtro de prefixo): {len(raw_candidates)}")

    filtered = [w for w in raw_candidates if w.lower().startswith(KNOWN_PREFIX)]
    print(f"[*] candidatos com prefixo '{KNOWN_PREFIX}': {len(filtered)}")

    tested = 0
    algos = hash_fns()
    for base in filtered:
        for cand in mutate(base):
            tested += 1
            for algo_name, fn in algos.items():
                if fn(cand) == TARGET_HASH:
                    print(f"\n[SUCESSO] senha encontrada!")
                    print(f"  senha    : {cand!r}")
                    print(f"  algoritmo: {algo_name}")
                    print(f"  tentativas: {tested}")
                    return cand, algo_name

    print(f"\n[FALHA] nenhuma senha encontrada entre {tested} tentativas "
          f"({len(algos)} algoritmos testados: {', '.join(algos)}).")
    return None, None


if __name__ == "__main__":
    main()
```

5.2. colab_quebra_senha.py

Etapa 1: força bruta por máscara na GPU (Google Colab) que encontrou a senha

```
# Quebra da senha por forca bruta na GPU (Google Colab)
#
# A ideia foi usar a GPU gratis do Colab (Tesla T4) pra rodar o hashcat bem
# mais rapido do que no nosso notebook. Como a senha comeca com "gu" e a gente
# ja tinha descartado os tamanhos menores rodando no CPU/GPU local, aqui a
# gente focou na mascara de 10 minusculas depois do "gu".
#
# Isso aqui veio de um notebook do Colab (.ipynb), as linhas que comecam com
# "!" sao comandos de shell e o resto e Python. Pra rodar: Ambiente de
# execucao -> Alterar tipo -> GPU, e ir rodando celula por celula.

# ver qual GPU o Colab deu dessa vez
get_ipython().system('nvidia-smi')

# instala o hashcat
get_ipython().system('apt-get install -y hashcat')

# confere se ele enxerga a GPU (tem que aparecer a Tesla T4 no OpenCL)
get_ipython().system('hashcat -I')

# grava o hash alvo num arquivo
with open("hash.txt", "w") as f:
    f.write("befb7036fb77a8517e95d9dbb797031850fce2ecdb988e96d7f040a2608e87e4")

get_ipython().system('cat hash.txt')

# monta o Drive pra salvar o progresso (se a sessao do Colab cair, da pra retomar)
from google.colab import drive
drive.mount('/content/drive')

get_ipython().system('mkdir -p /content/drive/MyDrive/hashcat_progresso')
get_ipython().system('ls /content/drive/MyDrive/hashcat_progresso')

# ataque de mascara: "gu" + 10 minusculas (?l)
#   -m 1400  -> SHA-256
#   -a 3     -> mascara (forca bruta)
#   -O       -> kernel otimizado (mais rapido)
#   -w 3     -> workload alto
#   --session + --restore-file-path -> salva o estado pra conseguir retomar
get_ipython().system('hashcat -m 1400 -a 3 hash.txt "gu?l?l?l?l?l?l?l?l?l" -O -w 3 -d 1 '
                     '--session=gpu_colab '
'--restore-file-path=/content/drive/MyDrive/hashcat_progresso/gpu_colab.restore')
```

5.3. etapa1_descobrir_senha.sh

Etapa 1: roteiro de força bruta/dicionário (hashcat)

```
#!/usr/bin/env bash
# Etapa 1 - descoberta da senha de key_for_rsa_public.hash
# Precisa de hashcat, python3 e as wordlists (wbrazilian, wportuguese,
# cracklib-runtime, john). Rodar os blocos em ordem; os marcados [FALHOU]
# sao as tentativas que nao deram certo.
set -e
cd "$(dirname "$0")"

echo "### 1) Dar uma olhada nos arquivos de entrada ###"
cd /home/eduarda/Documentos/CRIPTO
file message.en key_for_message.en key_public.en key_for_rsa_public.hash
echo "--- key_for_message.en (chave cifrada, RSA?) ---"
cat key_for_message.en
echo
echo "--- key_for_rsa_public.hash (hash da senha) ---"
cat key_for_rsa_public.hash
echo
H=$(tr -d '\n' < key_for_rsa_public.hash)
echo "tamanho do hash: ${#H} caracteres hex -> $(( ${#H} / 2 )) bytes (= digest de 256 bits)"

echo
echo "### 2) Localizar wordlists disponiveis no sistema ###"
dpkg -L wbrazilian 2>/dev/null | grep dict
dpkg -L wportuguese 2>/dev/null | grep dict
dpkg -L cracklib-runtime 2>/dev/null | grep dict
ls -la /usr/share/john/password.lst

echo
echo "### 3) Ataque de dicionario (script Python, prefixo 'gu' + mutacoes) ###"
cd /home/eduarda/Documentos/CRIPTO/scripts
python3 -I crack_hash.py
# [FALHOU] nada bateu (46560 tentativas em sha256/sha3_256/blake2s)

echo
echo "### 4) Benchmark do hashcat (ver se da pra fazer forca bruta) ###"
hashcat -I            # lista os dispositivos (aqui so CPU)
hashcat -b -m 1400     # benchmark do SHA-256 (modo 1400)

echo
echo "### 5) Forca bruta com mascara: 'gu' + sufixo minusculas+digitos ###"
echo "--- 5a) tamanho total 1 a 5 [FALHOU - increment mal configurado] ---"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=1 --increment-max=5 \
  target_hash.txt 'gu?1?1?1?1?1' -o cracked_5a.txt --force || true
# deu ruim aqui: o --increment conta o tamanho TOTAL da mascara (contando
# o "gu" fixo), nao so o sufixo, entao acabou testando menos do que a gente
# pensava.

echo "--- 5b) tamanho total 2 a 7 (sufixo 0 a 5), minusculas+digitos [FALHOU] ---"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=2 --increment-max=7 \
  target_hash.txt 'gu?1?1?1?1?1' -o cracked_5b.txt --force || true

echo "--- 5c) tamanho total 8 (sufixo 6), minusculas+digitos [FALHOU] ---"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=8 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_5c.txt --force || true

echo
echo "### 6) Forca bruta com mascara: minusculas+MAIUSCULAS+digitos ###"
echo "--- 6a) tamanho total 2 a 7 [FALHOU] ---"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?u?d --increment --increment-min=2 --increment-max=7 \
  target_hash.txt 'gu?1?1?1?1?1' -o cracked_6a.txt --force || true

echo "--- 6b) tamanho total 8 [FALHOU - colidiu com instancia anterior do hashcat] ---"
# [FALHOU] da primeira vez rodamos esse junto com outro hashcat e deu
# "Already an instance running" - so da pra rodar um hashcat por vez.
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?u?d --increment --increment-min=8 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_6b.txt --force || true

echo
echo "### 7) Forca bruta com mascara: ASCII imprimivel completo (com simbolos) ###"
echo "--- 7a) tamanho total 2 a 7 [FALHOU] ---"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?a --increment --increment-min=2 --increment-max=7 \
  target_hash.txt 'gu?1?1?1?1?1' -o cracked_7a.txt --force || true
echo
```

```
echo "### 8) Verificar se algum arquivo cracked_*.txt tem resultado ###"
for f in cracked_5a.txt cracked_5b.txt cracked_5c.txt cracked_6a.txt cracked_6b.txt cracked_7a.txt;
    do
  if [ -s "$f" ]; then
    echo ">>> SUCESSO em $f:"
    cat "$f"
  fi
done
echo "(se nada apareceu acima, nenhuma faixa testada ainda encontrou a senha)"

echo
echo "### 9) Testar outros hashes de 256 bits (rapido, so pra garantir) ###"
echo "--- 9a) SHA3-256 (modo hashcat 17400) ---"
hashcat -m 17400 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=2 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_9a.txt --force || true
hashcat -m 17400 -a 3 -O --potfile-disable \
  -1 ?l?u?d --increment --increment-min=2 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_9a2.txt --force || true

echo "--- 9b) Keccak-256 (modo hashcat 17800, pre-padronizacao do SHA3) ---"
hashcat -m 17800 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=2 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_9b.txt --force || true
hashcat -m 17800 -a 3 -O --potfile-disable \
  -1 ?l?u?d --increment --increment-min=2 --increment-max=8 \
  target_hash.txt 'gu?1?1?1?1?1?1' -o cracked_9b2.txt --force || true
# BLAKE2s nao tem modo no hashcat, so deu pra testar pelo crack_hash.py (bloco 3)

echo
echo "### 10) Forca bruta SHA-256, tamanho total 9, minusculas+digitos ###"
echo "    (mais pesado - 36^7 combinacoes, uns 15-20 min so de CPU)"
hashcat -m 1400 -a 3 -O --potfile-disable \
  -1 ?l?d --increment --increment-min=9 --increment-max=9 \
  target_hash.txt 'gu?1?1?1?1?1?1?1' -o cracked_10.txt --force || true

echo
echo "### 11) Chutes manuais ligados ao tema (nomes do grupo, Biuro Szyfrow...) ###"
python3 -I - <<'PYEOF'
import hashlib
target = "befb7036fb77a8517e95d9dbb797031850fce2ecdb988e96d7f040a2608e87e4"
candidatos = [
    "gu14", "GU14", "grupo14senha", "guBiuroSzyfrow", "guszyfrow",
    "guRejewski", "guEnigma", "guCriptoanalise", "gueduarda", "gujulia",
    "gubrenda", "gurosa", "gusilva", "gusouza",
]
algos = {
    "sha256": lambda s: hashlib.sha256(s.encode()).hexdigest(),
    "sha3_256": lambda s: hashlib.sha3_256(s.encode()).hexdigest(),
    "blake2s": lambda s: hashlib.blake2s(s.encode()).hexdigest(),
}
achou = False
for c in candidatos:
    for nome, fn in algos.items():
        if fn(c) == target:
            print(f">>> SUCESSO: senha={c!r} algoritmo={nome}")
            achou = True
if not achou:
    print("Nenhum candidato tematico bateu com o hash.")
PYEOF

echo
echo "### 12) Verificar TODOS os arquivos cracked_* no final ###"
for f in cracked_*.txt; do
  [ -s "$f" ] && echo ">>> SUCESSO em $f:" && cat "$f"
done
echo "(se nada apareceu acima, repensar algoritmo/encoding e expandir mascara)"
```

5.4. etapa2_analise_estrutural.sh

Etapa 2: análise estrutural dos arquivos cifrados

```
#!/usr/bin/env bash
# Analise dos arquivos cifrados so olhando tamanho e cabecalho (sem a senha).
# Serve pra deduzir o modo de cada cifra antes de tentar decifrar.
set -e
cd /home/eduarda/Documentos/CRIPTO

echo "### message.en ###"
wc -c message.en
xxd message.en | head -2
echo "Cabecalho: 'Salted__' (8 bytes, magic do openssl enc) + salt (8 bytes) = 16 bytes"
CT=$(( $(wc -c < message.en) - 16 ))
echo "Ciphertext: $CT bytes | resto mod 16 = $(( CT % 16 ))"
echo ">> multiplo exato de 16 => cifra de bloco de 128 bits (AES?) com padding (CBC/ECB)"

echo
echo "### key_public.en (base64) ###"
wc -c key_public.en
base64 -d key_public.en > /tmp/key_public_decoded.bin
wc -c /tmp/key_public_decoded.bin
xxd /tmp/key_public_decoded.bin | head -2
B=$(wc -c < /tmp/key_public_decoded.bin)
CT2=$(( B - 16 ))
echo "Ciphertext: $CT2 bytes | resto mod 16 = $(( CT2 % 16 )) | resto mod 8 = $(( CT2 % 8 ))"
echo ">> NAO e multiplo de 8 nem 16 (157 e primo) => sem padding de bloco"
echo ">> indica modo STREAM (CTR/CFB/OFB), nao CBC/ECB"

echo
echo "### key_for_message.en (hex) ###"
H=$(tr -d '\n' < key_for_message.en)
echo "tamanho: ${#H} hex chars -> $(( ${#H} / 2 )) bytes -> $(( ${#H} / 2 * 8 )) bits"
echo ">> 48 bytes = 384 bits: tem cara de cifra RSA crua (o ciphertext tem o tamanho do modulo)"
echo ">> nossa aposta: e um modulo RSA bem curto (384 bits), da pra fatorar"
```

5.5. etapa2_decifrar_key_public.sh

Etapa 2: decifração automática de key_public.en

```
#!/usr/bin/env bash
# Etapa 2 - decifra key_public.en (precisa da senha da Etapa 1).
# Uso: ./etapa2_decifrar_key_public.sh "senha"
#
# O ciphertext tem 157 bytes (nao multiplo de 8/16), entao e modo de fluxo
# (CTR/CFB/OFB), sem padding. O script testa varias cifras de fluxo x KDF
# (legado/pbkdf2) x digest e ve qual produz uma chave publica PEM/DER valida.
set -u
cd "$(dirname "$0")"

if [ $# -lt 1 ]; then
  echo "Uso: $0 <senha>"
  exit 1
fi
SENHA="$1"
INFILE_B64="/home/eduarda/Documentos/CRIPTO/key_public.en"
OUTDIR="./tentativas_key_public"
mkdir -p "$OUTDIR"

# Decodifica o base64 uma vez so. (Usar "-a -A" junto com "-provider legacy"
# dava "error reading input file" nesta versao do openssl, entao decodificamos
# na mao e passamos o binario direto.)
INFILE="$OUTDIR/_key_public_decoded.bin"
base64 -d "$INFILE_B64" > "$INFILE"

# Cifras de fluxo / sem padding de bloco (bate com os 157 bytes, que nao sao multiplos de 8/16)
CIPHERS=(
  aes-128-ctr aes-192-ctr aes-256-ctr
  aes-128-cfb aes-192-cfb aes-256-cfb
  aes-128-cfb8 aes-192-cfb8 aes-256-cfb8
  aes-128-ofb aes-192-ofb aes-256-ofb
  chacha20
  rc4 rc4-40
  bf-cfb bf-ofb
  cast5-cfb cast5-ofb
  des-cfb des-ofb des-ede3-cfb des-ede3-ofb
  sm4-ctr sm4-cfb sm4-ofb
  camellia-128-ctr camellia-192-ctr camellia-256-ctr
  camellia-128-cfb camellia-256-cfb
  camellia-128-ofb camellia-256-ofb
  aria-128-ctr aria-256-ctr
  seed-cfb seed-ofb
)

DIGESTS=(sha256 sha1 md5 sha512)
ITERS=(1 1000 4096 10000 50000 100000 250000 600000)

acertos=0
tentativas=0

tentar() {
  local cipher="$1" ; shift
  local extra_args=("$@")
  local label
  label="${cipher}_$(echo "${extra_args[*]}" | tr ' /' '__')"
  local outfile="$OUTDIR/${label}.pem"
  tentativas=$((tentativas+1))
  openssl enc -d -"$cipher" -in "$INFILE" -pass "pass:$SENHA" \
    -provider legacy -provider default \
    "${extra_args[@]}" -out "$outfile" 2>/dev/null
  if [ ! -s "$outfile" ]; then
    return
  fi
  if grep -q "BEGIN.*PUBLIC KEY" "$outfile" 2>/dev/null; then
    echo ">>> SUCESSO (PEM): cifra=$cipher args='${extra_args[*]}' -> $outfile"
    cat "$outfile"
    acertos=$((acertos+1))
    return
  fi
  # chave DER comeca com 0x30 (SEQUENCE do ASN.1); confirma com o openssl
  primeiro_byte=$(xxd -p -l1 "$outfile" 2>/dev/null)
  if [ "$primeiro_byte" = "30" ]; then
    if openssl pkey -pubin -inform DER -in "$outfile" -noout 2>/dev/null; then
      echo ">>> SUCESSO (DER): cifra=$cipher args='${extra_args[*]}' -> $outfile"
      openssl pkey -pubin -inform DER -in "$outfile" -text -noout 2>/dev/null
      acertos=$((acertos+1))
      return
    fi
  fi
```

```
}

echo "[*] Testando ${#CIPHERS[@]} cifras x KDF legado/pbkdf2 x ${#DIGESTS[@]} digests..."

for cipher in "${CIPHERS[@]}"; do
  # KDF legado, sem passar -md (usa o digest padrao do openssl)
  tentar "$cipher"
  # KDF legado, variando -md
  for md in "${DIGESTS[@]}"; do
    tentar "$cipher" -md "$md"
  done
  # PBKDF2 (openssl >= 1.1.0), com -md e variando -iter
  for md in "${DIGESTS[@]}"; do
    for it in "${ITERS[@]}"; do
      tentar "$cipher" -pbkdf2 -md "$md" -iter "$it"
    done
  done
done

echo
echo "[*] Total de tentativas: $tentativas | Sucessos: $acertos"
if [ "$acertos" -eq 0 ]; then
  echo "[FALHA] Nenhuma combinacao de cifra/KDF/digest produziu uma chave PEM/DER valida."
  echo
  echo "[*] Top 5 saidas mais 'parecidas com texto' (proporcao de bytes imprimiveis),"
  echo "    pra olhar na mao - a certa fica com proporcao perto de 1.0:"
  for f in "$OUTDIR"/*.pem; do
    [ -s "$f" ] || continue
    total=$(wc -c < "$f")
    printaveis=$(tr -dc '[:print:]\n' < "$f" | wc -c)
    razao=$(awk -v p="$printaveis" -v t="$total" 'BEGIN{printf "%.3f", p/t}')
    echo "$razao $f"
  done | sort -rn | head -5 | while read -r razao f; do
    echo "  razao=$razao  $f"
  done
fi
```

5.6. etapa3_fatorar_rsa.py

Etapa 3: fatoração do módulo RSA e decifração de key_for_message.en

```
#!/usr/bin/env python3
"""
Etapa 3 - fatora o modulo RSA, recupera a chave privada e decifra
key_for_message.en.
Uso: python3 -I etapa3_fatorar_rsa.py --n <modulo> --e <expoente> [--c <hex>]

O modulo tem ~382 bits (curto), entao da pra fatorar sem hardware especial.
Tenta, da mais barata pra mais cara: factordb, divisao por tentativa, Fermat
(bom se p e q sao proximos), Pollard rho e Pollard p-1. Se nenhuma dessas
der conta, precisa de uma ferramenta de crivo (yafu/msieve). So usa stdlib.
"""
import argparse
import math
import random
import sys
import urllib.request
import urllib.error
import json


def is_probable_prime(n, k=20):
    if n < 2:
        return False
    for p in (2, 3, 5, 7, 11, 13, 17, 19, 23, 29, 31, 37):
        if n % p == 0:
            return n == p
    d = n - 1
    r = 0
    while d % 2 == 0:
        d //= 2
        r += 1
    for _ in range(k):
        a = random.randrange(2, n - 1)
        x = pow(a, d, n)
        if x == 1 or x == n - 1:
            continue
        for _ in range(r - 1):
            x = pow(x, 2, n)
            if x == n - 1:
                break
        else:
            return False
    return True


def trial_division(n, bound=2_000_000):
    print(f"[*] Tentando divisao por tentativa (limite {bound})...")
    if n % 2 == 0:
        return 2, n // 2
    f = 3
    while f <= bound and f * f <= n:
        if n % f == 0:
            return f, n // f
        f += 2
    return None


def fermat_factor(n, max_iter=5_000_000):
    print(f"[*] Tentando fatoracao de Fermat (ate {max_iter} iteracoes)...")
    a = math.isqrt(n)
    if a * a < n:
        a += 1
    for i in range(max_iter):
        if i % 500_000 == 0 and i > 0:
            print(f"    {i:,}/{max_iter:,} iteracoes...", flush=True)
        b2 = a * a - n
        b = math.isqrt(b2)
        if b * b == b2:
            p, q = a - b, a + b
            if p > 1 and q > 1:
                return p, q
        a += 1
    return None


def pollard_rho(n, max_tries=20, max_iter_per_try=2_000_000):
    print(f"[*] Tentando Pollard's rho ({max_tries} tentativas x ate "
          f"{max_iter_per_try:,} iteracoes cada)...")
    if n % 2 == 0:
```

```
        return 2, n // 2

    def f(x, c, n):
        return (x * x + c) % n

    for tentativa in range(max_tries):
        x = random.randrange(2, n - 1)
        y = x
        c = random.randrange(1, n - 1)
        d = 1
        it = 0
        while d == 1 and it < max_iter_per_try:
            x = f(x, c, n)
            y = f(f(y, c, n), c, n)
            d = math.gcd(abs(x - y), n)
            it += 1
            if it % 200_000 == 0:
                print(f"    tentativa {tentativa+1}/{max_tries}, "
                      f"{it:,} iteracoes...", flush=True)
        if d != 1 and d != n:
            return d, n // d
        print(f"    tentativa {tentativa+1}/{max_tries} esgotada sem sucesso.",
              flush=True)
    return None


def pollard_p_minus_1(n, bound=1_000_000):
    print(f"[*] Tentando Pollard p-1 (limite de suavidade {bound})...")
    a = 2
    for p in range(2, bound):
        a = pow(a, p, n)
        if a == 0:
            return None
        d = math.gcd(a - 1, n)
        if 1 < d < n:
            return d, n // d
    return None


def factordb_lookup(n):
    print("[*] Consultando factordb.com (banco publico de fatoracoes)...")
    try:
        url = f"http://factordb.com/api?query={n}"
        with urllib.request.urlopen(url, timeout=10) as resp:
            data = json.loads(resp.read().decode())
        status = data.get("status")
        factors = data.get("factors", [])
        if status in ("FF", "P", "PRP") and len(factors) >= 2:
            # factors: lista de [fator, multiplicidade]
            flat = []
            for fac, mult in factors:
                flat.extend([int(fac)] * int(mult))
            if len(flat) == 2:
                return flat[0], flat[1]
            # mais de 2 fatores (nao deveria acontecer p/ RSA valido) - agrupa
            p = flat[0]
            q = n // p
            return p, q
        print(f"    factordb: status={status}, sem fatoracao completa disponivel.")
    except (urllib.error.URLError, TimeoutError, OSError) as exc:
        print(f"    factordb: sem acesso ({exc}).")
    except Exception as exc:
        print(f"    factordb: erro ao interpretar resposta ({exc}).")
    return None


def factor_n(n):
    for strategy in (factordb_lookup, trial_division, fermat_factor,
                     pollard_rho, pollard_p_minus_1):
        result = strategy(n)
        if result:
            p, q = result
            if p * q == n and p > 1 and q > 1:
                return p, q
            print(f"    (resultado inconsistente de {strategy.__name__}, ignorando)")
    return None


def main():
    ap = argparse.ArgumentParser()
    ap.add_argument("--n", required=True, help="modulo RSA (hex com 0x ou decimal)")
    ap.add_argument("--e", type=int, default=65537, help="expoente publico (default 65537)")
    ap.add_argument("--c", help="ciphertext de key_for_message.en (hex) - se passado, "
```

```
                                "decifra automaticamente apos fatorar N")
    args = ap.parse_args()

    n_str = args.n.strip()
    n = int(n_str, 16) if n_str.lower().startswith("0x") else int(n_str)
    e = args.e

    print(f"[*] N = {n}")
    print(f"[*] bits(N) = {n.bit_length()}")
    print(f"[*] e = {e}")
    print()

    result = factor_n(n)
    if not result:
        print("\n[FALHA] Nenhuma estrategia conseguiu fatorar N.")
        print("        Considerar instalar yafu/msieve/gmp-ecm/pari-gp para")
        print("        fatores maiores, ou revisar se N foi extraido corretamente.")
        sys.exit(1)

    p, q = result
    if p > q:
        p, q = q, p

    print(f"\n[SUCESSO] N = p * q")
    print(f"  p = {p}  ({p.bit_length()} bits)")
    print(f"  q = {q}  ({q.bit_length()} bits)")

    if not is_probable_prime(p) or not is_probable_prime(q):
        print("  [aviso] p ou q nao passou no teste de primalidade - revisar.")

    phi = (p - 1) * (q - 1)
    try:
        d = pow(e, -1, phi)
    except ValueError:
        print("  [ERRO] e nao e invertivel mod phi(n) - e e phi(n) nao sao coprimos.")
        sys.exit(1)

    print(f"\n  phi(N) = {phi}")
    print(f"  d (expoente privado) = {d}")
    print(f"\n  Chave privada RSA recuperada: (n={n}, e={e}, d={d})")

    if not args.c:
        print("\n  Use --c <hex de key_for_message.en> pra decifrar automaticamente,")
        print("  ou faca manualmente:")
        print("    c = int(open('key_for_message.en').read().strip(), 16)")
        print("    m = pow(c, d, n)")
        print("    plaintext_bytes = m.to_bytes((m.bit_length()+7)//8, 'big')")
        return

    c_str = args.c.strip()
    c = int(c_str, 16) if not c_str.lower().startswith("0x") else int(c_str, 16)
    nbytes = (n.bit_length() + 7) // 8  # tamanho do modulo em bytes

    print(f"\n[*] Decifrando key_for_message.en (RSA textbook, sem padding assumido)...")
    m = pow(c, d, n)
    plaintext = m.to_bytes(nbytes, "big")  # preserva zeros a esquerda do tamanho do modulo
    plaintext_stripped = plaintext.lstrip(b"\x00")

    print(f"  bytes (com zeros a esquerda do tamanho do modulo, {nbytes} bytes):")
    print(f"    hex: {plaintext.hex()}")
    print(f"  bytes (sem zeros a esquerda, {len(plaintext_stripped)} bytes):")
    print(f"    hex: {plaintext_stripped.hex()}")
    try:
        texto = plaintext_stripped.decode("utf-8")
        print(f"  como texto UTF-8: {texto!r}")
        print("\n  >>> Esse texto e provavelmente a SENHA usada em 'openssl enc -k'")
        print("      para cifrar message.en (ele tem cabecalho 'Salted__', ou seja,")
        print("      foi cifrado por senha, nao por chave bruta).")
        print(f"      Proximo passo: ./etapa4_decifrar_message.sh {texto!r}")
    except UnicodeDecodeError:
        print("  [aviso] nao decodifica como UTF-8 limpo - pode ter padding")
        print("          (OAEP/PKCS1) que nao foi removido, ou ser bytes binarios")
        print("          de chave+IV em vez de uma senha texto. Inspecionar o hex acima.")


if __name__ == "__main__":
    main()
```

5.7. etapa4_decifrar_message.sh

Etapa 4: decifração automática de message.en

```
#!/usr/bin/env bash
# Etapa 4 - decifra message.en com a senha recuperada no RSA (Etapa 3).
# Uso: ./etapa4_decifrar_message.sh "senha"
#
# message.en e binario (Salted__ + salt), ciphertext de 1520 bytes, multiplo
# de 16 -> cifra de bloco com padding (CBC/ECB). Testa varias cifras de bloco
# x KDF x digest e marca sucesso quando a saida tem os nomes do grupo (o
# enunciado garante que a mensagem contem os nossos nomes).
set -u
cd "$(dirname "$0")"

if [ $# -lt 1 ]; then
  echo "Uso: $0 <senha>"
  exit 1
fi
SENHA="$1"
INFILE="/home/eduarda/Documentos/CRIPTO/message.en"
OUTDIR="./tentativas_message"
mkdir -p "$OUTDIR"

# Cifras de bloco com padding (bate com os 1520 bytes, multiplo de 16)
CIPHERS=(
  aes-128-cbc aes-192-cbc aes-256-cbc
  aes-128-ecb aes-192-ecb aes-256-ecb
  des-cbc des-ecb des-ede3-cbc des-ede3-ecb des-ede-cbc des-ede-ecb
  bf-cbc bf-ecb
  cast5-cbc cast5-ecb
  camellia-128-cbc camellia-192-cbc camellia-256-cbc
  camellia-128-ecb camellia-192-ecb camellia-256-ecb
  aria-128-cbc aria-192-cbc aria-256-cbc
  aria-128-ecb aria-192-ecb aria-256-ecb
  sm4-cbc sm4-ecb
  seed-cbc seed-ecb
  rc2-cbc rc2-ecb
)

DIGESTS=(sha256 sha1 md5)

# Nomes dos integrantes - se aparecerem na saida, achamos a combinacao certa
NOMES_REGEX="eduarda|julia|brenda|rosa|silva|santos|souza"

acertos=0
tentativas=0

tentar() {
  local cipher="$1" ; shift
  local extra_args=("$@")
  local label
  label="${cipher}_$(echo "${extra_args[*]}" | tr ' /' '__')"
  local outfile="$OUTDIR/${label}.txt"
  tentativas=$((tentativas+1))
  openssl enc -d -"$cipher" -in "$INFILE" -pass "pass:$SENHA" \
    "${extra_args[@]}" -out "$outfile" 2>/dev/null
  if [ -s "$outfile" ] && grep -qiE "$NOMES_REGEX" "$outfile" 2>/dev/null; then
    echo ">>> SUCESSO: cifra=$cipher args='${extra_args[*]}' -> $outfile"
    echo "--- conteudo ---"
    cat "$outfile"
    echo "--- fim ---"
    acertos=$((acertos+1))
  fi
}

echo "[*] Testando ${#CIPHERS[@]} cifras x KDF legado/pbkdf2 x ${#DIGESTS[@]} digests..."

for cipher in "${CIPHERS[@]}"; do
  tentar "$cipher"
  for md in "${DIGESTS[@]}"; do
    tentar "$cipher" -md "$md"
  done
  for md in "${DIGESTS[@]}"; do
    tentar "$cipher" -pbkdf2 -md "$md"
  done
done

echo
echo "[*] Total de tentativas: $tentativas | Sucessos: $acertos"
if [ "$acertos" -eq 0 ]; then
  echo "[FALHA] Nenhuma combinacao produziu uma saida com os nomes esperados."
  echo "         Verificar se a senha/chave recuperada da Etapa 3 esta correta,"
```

```
  echo "         ou se key_for_message.en usa padding RSA (OAEP/PKCS1) que"
  echo "         precisa ser removido antes de usar os bytes como senha."
fi
```

6. Evidências de execução

Para comprovar que o trabalho foi de fato executado por nós, reunimos os registros brutos que as próprias ferramentas geraram durante a resolução. Os arquivos completos acompanham a entrega na pasta `evidencias_execucao/`.

6.1. Fatoração do RSA (yafu / SIQS)

A fatoração do módulo de 382 bits foi a etapa mais custosa: rodou por cerca de 11 horas num notebook com processador Intel Core i5-1235U (12 threads, AVX2). O trecho abaixo do log (`evidencias_execucao/factor.log`) registra o início na noite de 27/10 e o término na manhã de 28/10, com os dois primos recuperados ao final, exatamente o p e o q da seção 3.2:

```
10/27/26 19:13:31  starting SIQS on c115: 884448668806...83397028207
10/27/26 19:13:32  n = 117 digits, 386 bits
10/27/26 19:13:32  factor base: 298928 primes (max prime = 8924137)
10/27/26 19:13:32  using AVX2 enabled 32k sieve core
10/27/26 19:13:32  ==== sieving started (12 threads) ====
10/28/26 06:17:06  299844 relations found: 68714 full + 231130 from 5719221 partial
10/28/26 06:17:06  QS elapsed time = 39814.71 seconds
10/28/26 06:19:17  lanczos halted after 4575 iterations
10/28/26 06:19:18  prp58 = 30359271311154461518304651992102700956343898439652595494 41
10/28/26 06:19:18  prp58 = 29132737072042239871387085638859985911188525961047621213 27
10/28/26 06:19:18  SIQS elapsed time = 39946.24 seconds  (~11,1 h)
```

O arquivo `evidencias_execucao/siqs_progresso_amostra.txt` traz uma amostra do progresso ao vivo do peneiramento, com o número de relações subindo de 0 até as ~299 mil necessárias ao longo da madrugada (a ~145 relações por segundo). O estado completo do peneiramento (arquivo `siqs.dat`, de ~405 MB) não foi anexado por causa do tamanho, mas os logs reproduzem toda a linha do tempo.

6.2. Quebra da senha (hashcat na GPU)

A senha foi encontrada rodando o hashcat numa GPU Tesla T4 do Google Colab (notebook do apêndice 5.3), testando a máscara “gu” + 10 minúsculas. O acerto saiu em 52 segundos. O notebook, com as saídas de execução salvas, está disponível em:

`https://colab.research.google.com/drive/1Ew62xDPIvDcw4M2rQCV2qTLXA4SIf62N`